

Universidad Mariano Gálvez de Guatemala

Ingeniería en sistemas de computación



Tarea 6 - Colecciones y Excepciones

Angel Estuardo Campos Santay 9941-25-4809

Catedrático: Ing. David Alvarez

Cuarto ciclo

Sección C

Guatemala, 7 de septiembre del 2026

Análisis previo

- ¿Qué colección utilizaría para almacenar todos los productos comprados y conservar su orden de ingreso?

Utilizaría un ArrayList. Esta colección es la más adecuada porque trabaja como una lista dinámica que mantiene el orden exacto en el que se van agregando los elementos. Además, permite almacenar múltiples objetos de tipo Producto, incluso si comparten el mismo nombre o categoría, lo cual es necesario para guardar el registro histórico de las compras.

- ¿Qué colección utilizaría para almacenar las categorías sin repetirlas?

Utilizaría un HashSet. La propiedad principal de esta colección es que no permite elementos duplicados. Al momento de ir registrando los productos y guardar sus categorías en el HashSet, este se encarga automáticamente de descartar las que ya existen, asegurando que tengamos un listado limpio de categorías únicas sin necesidad de hacer validaciones manuales.

- ¿Qué estructura utilizaría para relacionar cada categoría con el total gastado en ella?

Utilizaría un HashMap. Esta estructura funciona organizando la información en pares de llave y valor (Key-Value). En este caso, la llave (Key) sería el nombre de la categoría en texto (String) y el valor (Value) sería el monto total acumulado en dinero (Double). De esta forma queda asociada directamente cada categoría con su gasto correspondiente.

- Una familia necesita buscar rápidamente cuánto ha gastado en una categoría específica, por ejemplo "Alimentos". ¿Cuál de las colecciones estudiadas sería más apropiada y por qué?

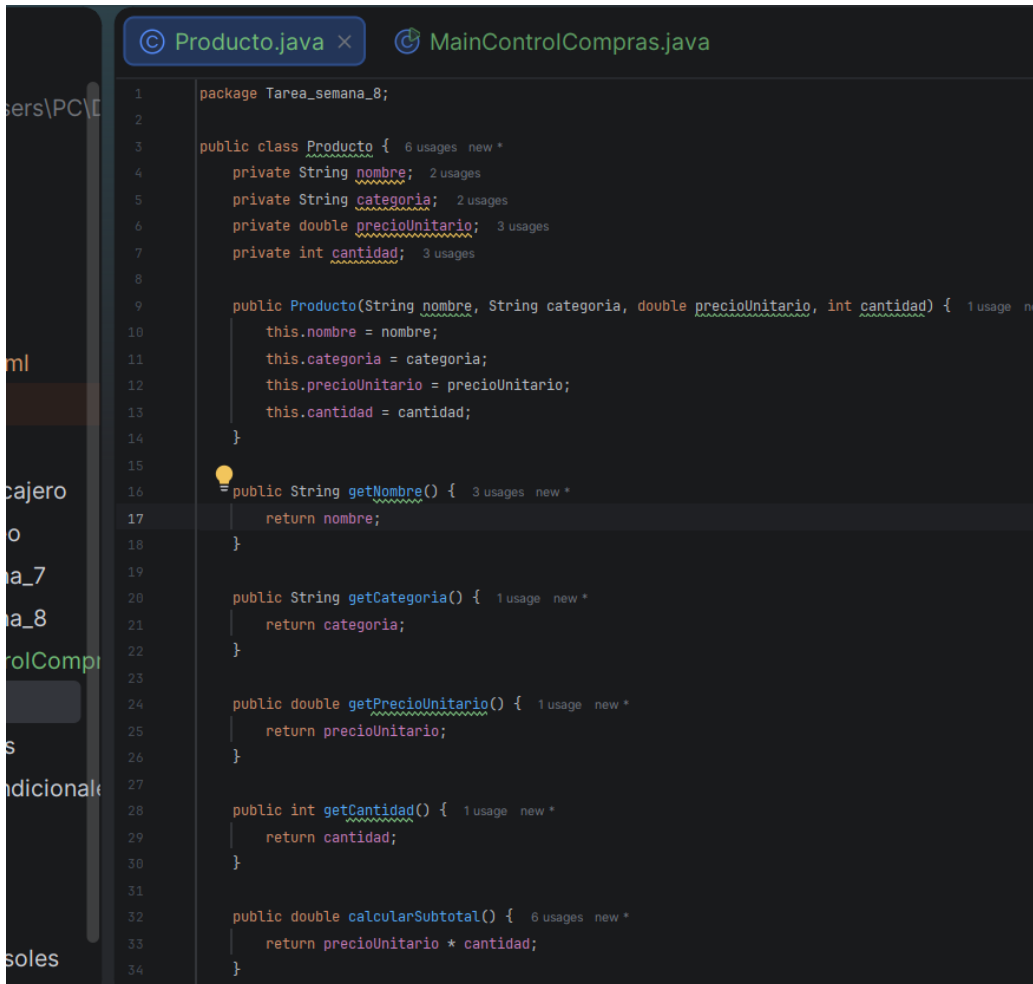
La colección más apropiada es el HashMap. A diferencia de una lista donde habría que recorrer elemento por elemento hasta encontrar la categoría deseada, el HashMap permite acceder al dato de forma directa utilizando la clave mediante el método `.get("Alimentos")`. Esto hace que la búsqueda sea inmediata y muy eficiente.

- ¿Por qué no sería conveniente utilizar una sola colección para resolver todas las necesidades anteriores?

No sería conveniente porque cada colección tiene una función y estructura especializada. Si intentáramos usar solo un ArrayList, tendríamos que escribir mucho código extra para evitar duplicados y hacer búsquedas lentas recorriendo toda la lista. Si usáramos solo un HashSet, perderíamos el orden de compra y los datos repetidos. Y si usáramos únicamente un HashMap, se nos complicaría guardar el historial de compras individuales de un mismo tipo. Combinar las tres colecciones aprovecha lo mejor de cada una y hace que el programa sea más eficiente y ordenado.

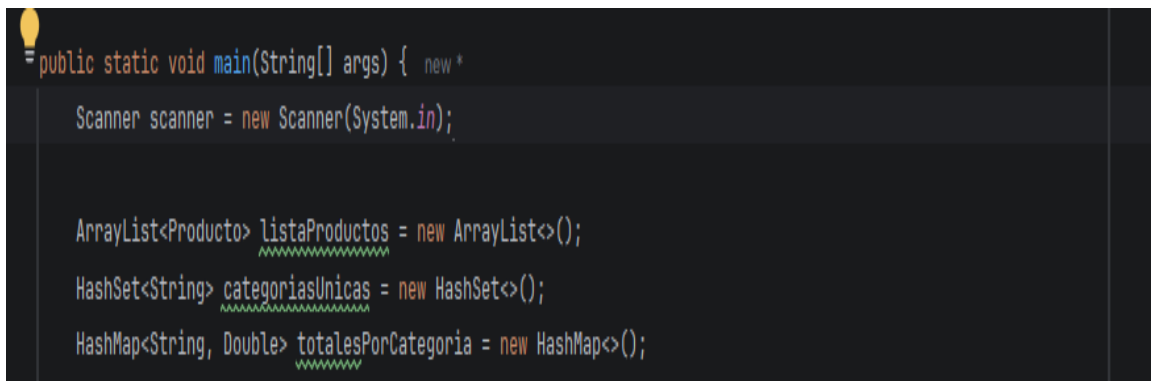
Evidencias

Código fuente de clase Producto:



```
1 package Tarea_semana_8;
2
3 public class Producto { 6 usages new *
4     private String nombre; 2 usages
5     private String categoria; 2 usages
6     private double precioUnitario; 3 usages
7     private int cantidad; 3 usages
8
9     public Producto(String nombre, String categoria, double precioUnitario, int cantidad) { 1 usage new *
10         this.nombre = nombre;
11         this.categoria = categoria;
12         this.precioUnitario = precioUnitario;
13         this.cantidad = cantidad;
14     }
15
16     = public String getNombre() { 3 usages new *
17         return nombre;
18     }
19
20     public String getCategoria() { 1 usage new *
21         return categoria;
22     }
23
24     public double getPrecioUnitario() { 1 usage new *
25         return precioUnitario;
26     }
27
28     public int getCantidad() { 1 usage new *
29         return cantidad;
30     }
31
32     public double calcularSubtotal() { 6 usages new *
33         return precioUnitario * cantidad;
34     }
```

declaraciones y utilización de ArrayList, HashSet y HashMap



```
1 = public static void main(String[] args) { new *
2
3     Scanner scanner = new Scanner(System.in);
4
5
6     ArrayList<Producto> listaProductos = new ArrayList<>();
7
8     HashSet<String> categoriasUnicas = new HashSet<>();
9
10    HashMap<String, Double> totalesPorCategoria = new HashMap<>();
```

Captura de la prueba con cinco productos válidos.

Run MainControlCompras

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2\lib\idea_rt.jar=60741" -Dfile.encoding=
=== REGISTRO DE COMPRAS DEL HOGAR ===
Debe registrar un minimo de 5 productos validos.

--- Ingrese Producto #1 ---
Nombre del producto: Jamón
Categoria: Embutido
Precio unitario: 12
Cantidad: 3
¡Producto registrado con éxito!

--- Ingrese Producto #2 ---
Nombre del producto: Leche
Categoria: Lacteos
Precio unitario: 20
Cantidad: 9
¡Producto registrado con éxito!

--- Ingrese Producto #3 ---
Nombre del producto: Tomates
Categoria: Verduras
Precio unitario: 4
Cantidad: 28
¡Producto registrado con éxito!

--- Ingrese Producto #4 ---
Nombre del producto: arroz
Categoria: granos
Precio unitario: 6
Cantidad: 100
¡Producto registrado con éxito!

--- Ingrese Producto #5 ---
Nombre del producto: pollo
Categoria: carne
Precio unitario: 15
Cantidad: 8
¡Producto registrado con éxito!
```

```
=====
===== RESUMEN DE COMPRAS =====
=====

Jamón | Embutido | Q12.00 x 3 | Subtotal: Q36.00
Leche | Lacteos | Q20.00 x 9 | Subtotal: Q180.00
Tomates | Verduras | Q4.00 x 28 | Subtotal: Q112.00
arroz | granos | Q6.00 x 100 | Subtotal: Q600.00
pollo | carne | Q15.00 x 8 | Subtotal: Q120.00

Categorías registradas:
[Embutido, Lacteos, Verduras, granos, carne]

Total por categoría:
Embutido: Q36.00
Lacteos: Q180.00
Verduras: Q112.00
granos: Q600.00
carne: Q120.00

Productos registrados: 5
Total general: Q1048.00

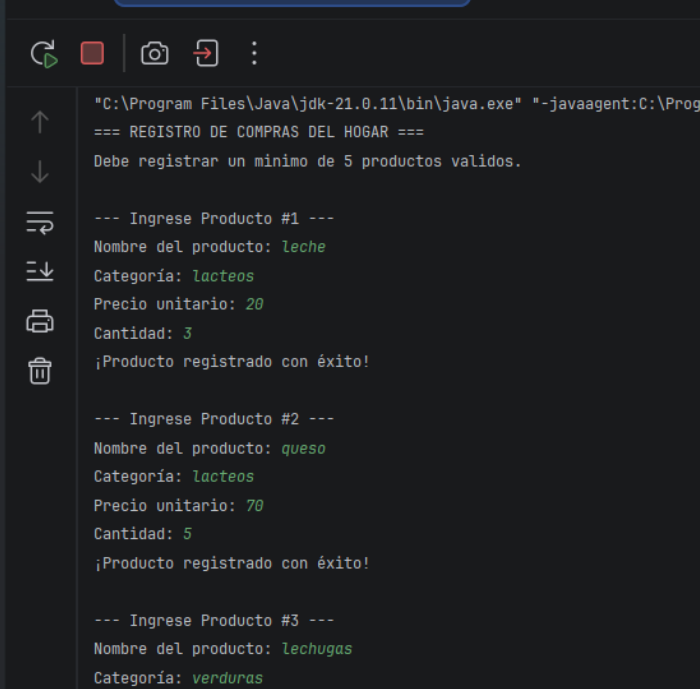
Producto con mayor gasto:
arroz - Q600.00

Producto con menor gasto:
Jamón - Q36.00

Categoría con mayor gasto:
granos - Q600.00

=====
Ingrese una categoría para consultar:
```

categoría repetida y el resultado sin duplicados.



```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\J...
=== REGISTRO DE COMPRAS DEL HOGAR ===

Debe registrar un minimo de 5 productos validos.

--- Ingrese Producto #1 ---
Nombre del producto: leche
Categoría: lacteos
Precio unitario: 20
Cantidad: 3
¡Producto registrado con éxito!

--- Ingrese Producto #2 ---
Nombre del producto: queso
Categoría: lacteos
Precio unitario: 70
Cantidad: 5
¡Producto registrado con éxito!

--- Ingrese Producto #3 ---
Nombre del producto: lechugas
Categoría: verduras
Precio unitario: 19
Cantidad: 4
¡Producto registrado con éxito!
```

```

--- Ingrese Producto #4 ---
Nombre del producto: zanahorias
Categoría: verduras
Precio unitario: 9
Cantidad: 8
¡Producto registrado con éxito!

--- Ingrese Producto #5 ---
Nombre del producto: maiz
Categoría: granos
Precio unitario: 500
Cantidad: 10
¡Producto registrado con éxito!

=====
===== RESUMEN DE COMPRAS =====
=====

leche | lacteos | Q20.00 x 3 | Subtotal: Q60.00
queso | lacteos | Q70.00 x 5 | Subtotal: Q350.00
lechugas | verduras | Q19.00 x 4 | Subtotal: Q76.00
zanahorias | verduras | Q9.00 x 8 | Subtotal: Q72.00
maiz | granos | Q500.00 x 10 | Subtotal: Q5000.00

Categorías registradas:
[verduras, lacteos, granos]

```

```

--- Ingrese Producto #5 ---
Nombre del producto: maiz
Categoría: granos
Precio unitario: 500
Cantidad: 10
¡Producto registrado con éxito!

=====
===== RESUMEN DE COMPRAS =====
=====

leche | lacteos | Q20.00 x 3 | Subtotal: Q60.00
queso | lacteos | Q70.00 x 5 | Subtotal: Q350.00
lechugas | verduras | Q19.00 x 4 | Subtotal: Q76.00
zanahorias | verduras | Q9.00 x 8 | Subtotal: Q72.00
maiz | granos | Q500.00 x 10 | Subtotal: Q5000.00

Categorías registradas:
[verduras, lacteos, granos]

Total por categoría:
verduras: Q148.00
lacteos: Q410.00
granos: Q5000.00

Productos registrados: 5
Total general: Q5558.00

Producto con mayor gasto:
maiz - Q5000.00
█

Producto con menor gasto:
leche - Q60.00

Categoría con mayor gasto:
granos - Q5000.00

=====
Ingrese una categoría para consultar:

```

Font

Precio igual o menor que cero.

```
Run MainControlCompras x

"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ I
=== REGISTRO DE COMPRAS DEL HOGAR ===
Debe registrar un minimo de 5 productos validos.

--- Ingrese Producto #1 ---
Nombre del producto: pollo
Categoría: carnes
Precio unitario: 0
Cantidad: 10
Producto no registrado: El precio debe ser mayor que cero.

--- Ingrese Producto #2 ---
Nombre del producto: |
```

Cantidad igual o menor que 0

```
--- Ingrese Producto #2 ---
Nombre del producto: uvas
Categoría: frutas
Precio unitario: 10
Cantidad: -1
Producto no registrado: La cantidad debe ser mayor que cero.

--- Ingrese Producto #3 ---
Nombre del producto: |
```

Búsqueda de categoría inexistente

Run MainControlCompras x



↑

↓

⇌

⇓

🖨

🗑

arroz | granos | Q19.00 x 300 | Subtotal: Q5700.00

manzanas | frutas | Q100.00 x 4 | Subtotal: Q400.00

lechugas | verduras | Q19.00 x 8 | Subtotal: Q152.00

frijol | granos | Q90.00 x 18 | Subtotal: Q1620.00

fresas | frutas | Q18.00 x 90 | Subtotal: Q1620.00

Categorías registradas:

[verduras, frutas, granos]

Total por categoría:

verduras: Q152.00

frutas: Q2020.00

granos: Q7320.00

Productos registrados: 5

Total general: Q9492.00

Producto con mayor gasto:

arroz - Q5700.00

Producto con menor gasto:

lechugas - Q152.00

Categoría con mayor gasto:

granos - Q7320.00

=====

Ingrese una categoría para consultar: *embutidos*

La categoría ingresada no se encuentra registrada.

Búsqueda de categoría existente


```
salchichitas | embutidos | Q6.00 x 60 | Subtotal: Q640.00
queso | lacteos | Q89.00 x 7 | Subtotal: Q623.00

Categorías registradas:
[lacteos, fruta, embutidos, granos]

Total por categoría:
lacteos: Q623.00
fruta: Q2240.00
embutidos: Q640.00
granos: Q4800.00

Productos registrados: 5
Total general: Q8303.00

Producto con mayor gasto:
meiz - Q4800.00

Producto con menor gasto:
queso - Q623.00

Categoría con mayor gasto:
granos - Q4800.00

=====
Ingrese una categoría para consultar: granos
Total gastado en granos: Q4800.00

Process finished with exit code 0
```

Resumen final del programa

```
Run MainControlCompras x
===== RESUMEN DE COMPRAS =====
=====
manzana | fruta | Q19.00 x 80 | Subtotal: Q1520.00
banano | fruta | Q8.00 x 90 | Subtotal: Q720.00
meiz | granos | Q40.00 x 120 | Subtotal: Q4800.00
salchichas | embutidos | Q8.00 x 80 | Subtotal: Q640.00
queso | lacteos | Q89.00 x 7 | Subtotal: Q623.00

Categorías registradas:
[lacteos, fruta, embutidos, granos]

Total por categoría:
lacteos: Q623.00
fruta: Q2240.00
embutidos: Q640.00
granos: Q4800.00

Productos registrados: 5
Total general: Q8303.00

Producto con mayor gasto:
meiz - Q4800.00

Producto con menor gasto:
queso - Q623.00

Categoría con mayor gasto:
granos - Q4800.00

=====
Ingrese una categoría para consultar: granos
Total gastado en granos: Q4800.00
```

Cálculo Manual de Subtotales y Total General

Para comprobar la exactitud de los algoritmos implementados en las colecciones de Java, se realiza la verificación matemática manual utilizando las fórmulas del programa.

Fórmula del Subtotal:

Subtotal= Precio unitario * cantidad

Fórmula del total general

$$\text{Total General} = \sum_{i=1}^n \text{Subtotal}_i$$

1. Cálculo Manual de dos Subtotales

- Producto 1: Leche
 - Datos: Precio Unitario = Q20.00 | Cantidad = 3
 - Operación: $20.00 \times 3 = 60.00$
 - Subtotal 1: Q60.00
- Producto 2: Queso
 - Datos: Precio Unitario = Q70.00 | Cantidad = 5
 - Operación: $70.00 \times 5 = 350.00$
 - Subtotal 2: Q350.00

2. Cálculo Manual del Total General

Sumando los subtotales de los 5 productos registrados en la prueba:

1. Leche: Q60.00
2. Queso: Q350.00
3. Lechugas: Q76.00 (19.00×4)
4. Zanahorias: Q72.00 (9.00×8)
5. Maíz: Q5000.00 (500.00×10)

Suma de Totales:

$$\text{Total General} = 60.00 + 350.00 + 76.00 + 72.00 + 5000.00 = \text{Q5558.00}$$

Reflexión final

- ¿Por qué fue necesario utilizar tres colecciones diferentes?

Fue necesario porque cada colección resuelve una necesidad distinta dentro del sistema. El ArrayList nos sirve para guardar el historial completo de los productos en el orden en que se compraron, el HashSet nos ayuda a obtener una lista limpia de categorías sin repetidas de forma automática, y el HashMap nos permite asociar cada categoría con su total de dinero acumulado para hacer búsquedas rápidas.

- ¿Qué ventaja proporcionó el HashSet en comparación con almacenar las categorías únicamente en el ArrayList?

La ventaja principal es que el HashSet evita duplicados por su propia naturaleza. Si hubiéramos usado únicamente el ArrayList, tendríamos que haber escrito condiciones manuales para revisar si la categoría ya existía antes de agregarla. El HashSet simplifica el código y evita errores al filtrar automáticamente los nombres repetidos.

- ¿Qué ventaja proporcionó utilizar un HashMap para almacenar los totales por categoría?

La ventaja es la rapidez y la facilidad para consultar la información. Al guardar los datos en formato de clave y valor, podemos saber cuánto se gastó en cualquier categoría directamente por su nombre mediante el método `.get()`, sin necesidad de recorrer de nuevo toda la lista de productos ni hacer sumas adicionales.

- ¿Qué parte del programa requirió mayor razonamiento?

La parte que requirió mayor razonamiento fue la lógica dentro del ciclo de registro, específicamente al actualizar los valores dentro del HashMap. Había que verificar si la categoría ya tenía un monto registrado para sumarle el nuevo subtotal o si era la primera vez que se ingresaba, utilizando la lógica del acumulador junto con las validaciones de datos numéricos.

- ¿Cómo podría utilizarse una aplicación similar en una situación real?

Una aplicación así se puede utilizar en la vida diaria para el control del presupuesto personal o familiar, ayudando a identificar en qué áreas se está gastando más dinero (como alimentos, entretenimiento o servicios). También podría servir de base para un sistema básico de inventario o facturación en un pequeño negocio para clasificar ventas por tipo de producto.